

The marketing site and the storefront

Two surfaces, one codebase, two entirely different audiences. One of them we design for ourselves and control completely. The other gets worn by fifty businesses we haven't met yet, and that constraint decides almost every choice in it.

For design **Scope** marketing site + public storefront **Not in scope** tenant admin dashboard

Market Nigeria · NGN only

The business underneath

WHY ANY OF THIS LOOKS THE WAY IT DOES

We sell a working online store to Nigerian small businesses — a real one, where money actually moves and stock actually decrements — and the target is roughly fifty of them a month.

Where we sit in that market is the single most useful thing to hold in your head, because we're between two competitors who are both easy to name. Below us, at around ₦25,000, is the WordPress-and-Elementor crowd selling a brochure site: four pages, a contact form, no cart. Above us, at ₦150,000 and up, are agencies selling bespoke work. We are closer to the brochure end on price and much closer to the agency end on what the thing actually does, and that gap is the entire pitch.

NOT YET FIXED

Our own price isn't settled, so nothing in this brief quotes one. Please design the pricing page around a figure that can change — a single prominent number with room to breathe, not a layout that only works at a particular character count or a comparison table with the number baked into the composition.

The economics only work because it is one application, not fifty. Every business is a row in a shared database, isolated by a tenant ID, served from one deployment. A tenant gets `theirstore.yourbrand.com` for free, or points their own domain at us. **Onboarding a client is a configuration task, not a build.** Everything in this brief follows from that: you are designing a template, once, and every design decision either survives fifty unknown brands or it doesn't ship.

One more piece of the model matters for the marketing copy. Customer payments never pass through an account we control — Paystack splits each transaction at settlement and the money lands directly in the client's own bank account. That isn't a technical footnote, it's the answer to the loudest objection a Nigerian business owner will have about handing their sales to a stranger's website, and it deserves real estate on the marketing site.

	MARKETING SITE	STOREFRONT
Lives at	yourbrand.com	{store}.yourbrand.com or the tenant's own domain
Audience	A business owner currently selling in Instagram comments and WhatsApp DMs	That owner's customer, arriving from a link in a bio
The job	Convince them that a real store is worth paying for and that we won't touch their money	Get one product bought without friction and without triggering scam suspicion
Branding	Ours. Full freedom.	Theirs — but expressed through exactly one hex value, a logo, and a tagline
Designed	Once, for one client: us	Once, for fifty clients we haven't met
Auth	Store owners sign in (Clerk)	Never. Shoppers do not have accounts

The storefront is the harder brief and the more valuable one — it's the thing fifty clients look at every day and the thing their customers judge. The marketing site is the easier brief but it's what converts, so it can't be an afterthought either.

Surface A — the marketing site

ROOT DOMAIN

What we are actually competing against

Not other web design agencies. The real competitor is **doing nothing** — continuing to sell through DMs. That's free, it already works well enough, and it's familiar. So a page that leads with "get a beautiful website" argues against the wrong thing, because the seller doesn't feel the absence of a website. They feel specific, daily, nameable pain:

- Chasing payment screenshots and squinting at them to check they're real
- Orders buried in an inbox with three hundred other messages
- Working out a delivery fee by hand, in a DM, for every single customer
- No idea what's actually in stock without walking to the shelf
- Sending account numbers to strangers and hoping the transfer lands

That list is the copy spine of the home page. Every feature we have maps onto one of those pains — checkout replaces the screenshot chase, delivery zones replace the mental arithmetic, the orders list replaces the inbox. Lead with the pain, land the feature second.

Where the visitor comes from, and what that means

Almost entirely from Instagram, TikTok, or a WhatsApp forward. Two consequences:

1. **Mobile-first is not a nicety, it's the only view most people will ever see** — and often inside an in-app browser with a fat top and bottom chrome, so vertical space is tighter than a normal mobile viewport.
2. **The page will be screenshotted and forwarded.** That's genuinely the distribution mechanic. Design at least one screen — most likely the pricing block — so that a phone screenshot of it, with no surrounding context, still explains the offer and carries the domain.

Sections the home page needs

Hero

PLACEHOLDER ONLY

Show the product, not a stock photo of a smiling woman with a laptop. The most persuasive thing we own is a real storefront on a real phone. A device frame with an actual seeded store in it does more work than any illustration, and it costs nothing because the store already exists.

Headline should name the outcome in the seller's language — taking orders and getting paid — not the artifact ("a website").

Live demo store

NOT BUILT

A prominent link to a real, browsable, seeded storefront. This is the single highest-converting asset on the site and it already exists in the system — two demo tenants are seeded. Someone who adds a product to a cart on the demo has effectively pre-sold themselves. Treat it as a primary call to action sitting beside "Get started", not a small text link.

"Your money goes straight to your bank"

NOT BUILT

The trust objection, answered visually. Customer pays → Paystack splits → the client's own bank account. We are never a box in that diagram holding funds. This wants to be an actual drawn flow, not three bullet points, because the shape of the diagram is the argument: there's no reservoir in the middle.

Adjacent to it: the client keeps their own Paystack dashboard and can see their own money at any time. That detail defuses the follow-up objection.

What you get / What you operate

NOT BUILT

Two distinct ideas that often get merged and shouldn't be. *What you get* is the storefront — cart, checkout, coupons, delivery zones. *What you operate* is the admin dashboard, and it's underrated as a selling point: the client will be in it every day adding products and marking orders shipped. Show it. A screenshot of a clean orders table is a feature, because the alternative they're living with is a WhatsApp thread.

/pricing

ROUTE RESERVED, PAGE NOT BUILT

The base package is settled even though the price isn't, and its contents are worth stating plainly rather than hiding behind "contact us":

- Up to ~30 products
- One delivery zone structure, around three tiers
- One payment gateway
- One or two launch coupon codes
- Mobile storefront plus admin access
- One round of design revisions
- Seven days of post-launch bug support

Add-ons — extra products, courier integration, monthly maintenance, custom domain setup, SEO basics, catalogue sync — are where margin actually lives, so the pricing page needs a second tier of presentation for them. Not buried, not competing with the headline figure.

Worth designing the anchor explicitly: a brochure site next to what this one does. The comparison is what justifies the price, whatever we land on.

/sign-in · /sign-up

BUILT, USING CLERK'S OWN COMPONENTS

These render Clerk's hosted components, so we do not control their markup. Styling happens through Clerk's appearance API — you can set colors, radii, fonts and element-level overrides, but not restructure the form. Please design the surrounding page (the frame, the left panel, the copy) and give me a token set for the form rather than a pixel comp of it.

NOTE ON THE HEADER

The site header already switches between signed-out (Sign in / Sign up) and signed-in (Dashboard link + account button) states. Two states to design, not one.

Surface B — the storefront

TENANT SUBDOMAINS AND CUSTOM DOMAINS

Everything a tenant can change about their store's appearance is on this list, and the list is deliberately short.

FIELD	WHAT IT IS	WHAT YOU CAN RELY ON
logoUrl	One image, any host	Nothing. Expect square Instagram avatars, wide wordmarks, transparent PNGs, and 400px-wide JPEGs with white backgrounds. Currently rendered at a fixed 32px height, auto width.
primaryColor	One hex string	Nothing. See the next section — this is the central design problem.
name	Store name	Always present. Used as the logo fallback. Can be long.
tagline	Short line	Optional. Currently footer only.
contactEmail	Email	Optional. Footer.
whatsappNumber	Phone	Optional. Rendered as a wa.me link. High-value for trust.
storeAddress	Physical address	Exists in the data model, currently rendered nowhere . See trust section.
currency	NGN	Always NGN. Do not design a currency switcher.

HARD CONSTRAINT

There is no per-tenant stylesheet, no per-tenant font, no per-tenant layout, and no theme picker. One stylesheet serves every store; the tenant color is injected as a single CSS custom property. A design that needs a second layout variant to look right is a design we can't ship, because "reskin per client" is exactly the cost the whole architecture exists to avoid.

The pages

/ – catalogue

BUILT, MINIMAL

Today: a bare product grid. Two columns on mobile, three at medium, four at large. Square image, name, price, and an out-of-stock marker. No hero, no categories, no search, no sort, no pagination.

That's defensible for a thirty-product store and it's fast, which matters. But the first thing a store owner will ask for is some way to say something above the grid — a banner, a featured product, a line of copy. Worth deciding now whether that's in scope, because it changes the data model.

`src/components/pages/storefront/home/index.tsx`

/products/[slug]

BUILT, SINGLE IMAGE ONLY

Image, name, price, description, a quantity stepper, and add-to-cart. The data model stores an *array* of image URLs but the page renders only the first one — a gallery is unbuilt and probably should exist, since clothing and accessories are the dominant category and one photo doesn't sell a garment.

Descriptions are free text written by the tenant. Expect anything from empty to six paragraphs of unformatted prose with emoji. Design for both ends.

`src/components/pages/storefront/product-detail/index.tsx`

/cart

BUILT

Line items, quantity adjustment, remove, subtotal. The cart lives in the browser's local storage and is scoped per store, so two storefronts open in the same browser can't contaminate each other.

Important consequence for you: **the cart cannot show a delivery fee**, because delivery zone isn't known until checkout. The subtotal here is not the final number and the design has to be honest about that without frightening anyone.

`src/components/pages/storefront/cart/index.tsx`

/checkout

BUILT

Single page, no steps. Fields, in order:

- Name, email, phone
- Delivery method — "Deliver to my address" or "Pick up in store (free)"
- Delivery zone, a select, only when delivering
- Delivery address, only when delivering
- Coupon code, optional

A live order summary sits alongside. It is explicitly *indicative* — the server recalculates every price, fee and discount from the database, and a coupon's real value is only confirmed at the payment step. The current copy handles this with a small grey line; if you can make that clearer without adding anxiety, that's a genuine improvement.

Guest checkout only. There is no account creation here, ever, and no "save my details for next time".

`src/components/pages/storefront/checkout/index.tsx`

Payment

PAYSTACK'S UI — NOT OURS

Payment happens in a Paystack overlay on top of our checkout page. The customer never leaves. We don't control that interface and shouldn't try to — Nigerian shoppers *recognise* it, and that recognition is a trust asset. Don't disguise it.

What you do design is the moment before (the pay button and its loading state) and the moment after (the customer lands back on our page). One firm rule inherited from the codebase: **payment errors appear inline, next to the pay button — never as a toast**. A toast that a customer misses at the moment money is involved is a support ticket at best.

/order/[reference]

BUILT — READ THE STATES SECTION

The confirmation page, and the most product-sensitive screen in the whole storefront. It has three states and a rule about a fourth. Covered below in detail — please don't design it from this summary alone.

`src/components/pages/storefront/order-confirmation/index.tsx`

Shown when a hostname matches no active store — a mistyped subdomain, a domain pointed at us before the store is configured, an expired client. Currently two lines of centred text.

Small page, real decision: is this neutral, or is it a soft platform surface that says who we are and links to the marketing site? Someone landing here is already curious about a Nigerian online store.

The brand-variable problem

THE CENTRAL DESIGN CONSTRAINT

A tenant supplies one colour. We apply it everywhere the design says "brand". We have no control over what they pick, and no design review before it goes live.

This is the constraint that most changes how you'd normally work. Below is the same component with five plausible tenant colours. Three are fine. Two produce white text that is genuinely unreadable, and both are colours a real Nigerian fashion or beauty brand would plausibly choose.

Identical markup · five tenant values · white foreground assumed

 #0E6B4F	 #7A2E8C	 #D81B60	 #7CE38B	 #F2D024
 3	 3	 3	 3	 3
 Add to cart	 Add to cart	 Add to cart	 Add to cart	 Add to cart
Fine	Fine	Borderline	Unreadable	Unreadable

Two design responses, and I'd like your call on both:

1. **Compute the foreground.** Rather than hard-coding white text on the brand colour, derive black or white from the colour's luminance so contrast always passes. Cheap to implement, removes the worst failures, but means a tenant's button sometimes has black text and sometimes white — the design has to look intentional either way.
2. **Spend the colour sparingly.** The more surface area the brand colour occupies, the more a bad value can wreck. If it's an accent — the cart badge, the primary button, a focus ring, an active state — the store still looks composed with a terrible hex. If it's a full-bleed header, one bad tenant means fifty support messages that all say "my site looks broken".

My strong recommendation is that the storefront's own palette does the heavy lifting — a neutral, quiet, well-made ground — and the tenant colour is a controlled accent on top. That reads as a professional template that the brand sits inside, rather than a template desperately pretending to be bespoke. Worth noting the second-order benefit too: a quiet ground makes *product photography* the loudest thing on the page, which is what actually sells.

RELATED, UNSOLVED

The logo has the same problem in a different shape. Fixed-height rendering means a wide wordmark becomes tiny and a square avatar becomes a postage stamp. Please specify a slot that handles both, plus the no-logo fallback (currently the store name as plain text, which is honestly fine and could be made good).

Trust is the design problem

THE THING THIS MARKET ACTUALLY TURNS ON

A shopper clicks a link in an Instagram bio and arrives at `somestore.yourbrand.com`. They have never heard of this store. Nigerian online shopping carries a real and rational fraud-suspicion baseline, and that shopper is asking one question before they'll consider a second: *is this a real business or am I about to send money to nobody?*

Conversion here is far more a function of credibility than of aesthetics. Every one of these is a design lever, not a content afterthought:

- **Visible, human contact.** Email and a WhatsApp link are in the footer already. WhatsApp especially — being able to message a person before paying is the difference between a real shop and a scam page. Consider whether it earns a more prominent position than the footer.
- **A physical address.** The field exists in the data model and is rendered nowhere. A Lagos street address does an enormous amount of credibility work for near-zero design cost. This is the cheapest win in the brief.
- **No surprise costs.** Delivery fee only becomes knowable at checkout, which is the classic abandonment trigger. At minimum, signal earlier that delivery is charged by zone and that pickup is free. If we can surface a "from ₦X" figure on the cart, better.
- **Pickup as a visible option.** It reads as logistics but functions as trust — "I can just go and collect it" defuses the fear of paying into the void. Don't bury it behind a delivery-first default.
- **A human order number.** Orders carry a readable per-store sequence number, not just an internal ID. Show that. "Order #14" is something a customer can quote in a WhatsApp message; a random string isn't.
- **Paystack, named.** Shoppers recognise and trust it. Saying payments are processed by Paystack borrows credibility we haven't earned yet.

Speed is a design constraint

NOT A DEVELOPER CONCERN

Page speed is written into what we sell — it's an explicit differentiator against the plugin-heavy WordPress builds we're priced above. The storefront has a hard target of a Lighthouse mobile score of 90 or better, measured on a seeded store. That's an acceptance criterion, so a design that can't hit it doesn't ship.

Practically, for your side of the work:

- No hero video, no autoplay background media on the storefront.
- One typeface family. Two weights if we can manage it. Every additional weight is real milliseconds on a 3G connection in Lagos.
- No carousel or animation libraries. CSS transitions are free; JavaScript ones aren't.
- An image budget per screen — tenants upload phone photos and won't optimise them.
- The catalogue and product pages are rendered on the server for search indexing; the cart is browser-only. That split is fixed.

This constraint is a gift, honestly. It rules out most of the ways a storefront template gets bloated and pushes the design toward exactly the restraint the brand-variable problem already demands.

States and edges

WHERE THE REAL WORK IS

The confirmation page, in detail

After paying, the customer lands on the order page while two things race: our browser-side check with Paystack, and a webhook arriving directly from Paystack to our server. Either can confirm the order. The second one might land seconds later.

verifying

Default on arrival. We're checking. Short, usually under two seconds.

paid

Confirmed. Show the order number and total. The cart clears here.

pending

We couldn't confirm yet. Copy says the order is *being confirmed* — reassuring, not alarming.

failed

Does not exist and must not be designed.

NEVER

A failure to confirm is **not** a failed payment. The webhook is the trusted path and it may not have landed yet. Telling someone their payment failed when it actually succeeded is by far the worse error — they will pay twice, or they'll be furious, or both. The `pending` state must read as "we've got this, check your email", never as "something went wrong".

This is the clearest example in the project of a design decision that is really a product decision about money. The visual difference between a reassuring pending state and an alarming one is the difference between a customer who waits and a customer who disputes a charge.

Empty and edge states, all of which are routine

STATE	HOW OFTEN	NOTE
Store with zero products	Every tenant, day one	Not an edge case — it's the first impression of every store we launch, and the client will look at it before their customers do. Currently one grey sentence. Deserves better.
Product with no image	Common	Currently renders an empty grey square. Needs a designed placeholder.
Out of stock	Common	Needed on both the grid and the product page, with add-to-cart disabled.
Empty cart	Common	Reachable directly by URL. Needs a route back to the catalogue.
Invalid or expired coupon	Common	Validated server-side and rate-limited, because coupon codes are otherwise guessable. Error appears at checkout.
Last item sold while checking out	Rare, high stakes	The order is still marked paid and flagged internally — we never drop a payment that actually happened. But the customer needs telling, gently, that one item will be sorted out directly. No screen exists for this yet.
Very long store or product names	Common	Tenants write their own copy with no character limits enforced in the UI.

What this will never have

SO YOU DON'T DESIGN IT

These aren't "later" — they're structurally absent, and designing around them would create expectations we'd then have to break.

- **Customer accounts.** There is no customer record in the database at all. Checkout is guest-only, forever. No sign-in on the storefront, no "my orders", no wishlist, no saved addresses.
- **Multiple currencies.** NGN only.
- **Reviews, ratings, related products, recommendations.** None of these are in the data model.
- **Live courier tracking.** Sold as a paid add-on, not part of the base product. Delivery is flat-rate by zone, plus free pickup.
- **Per-tenant themes or layout choices.** Covered above, but worth repeating because it's the one clients will ask for most.

Decisions I need from you

BLOCKING OR NEAR-BLOCKING

QUESTION	WHY IT'S BLOCKING
How much surface does the tenant colour get?	Determines whether one bad hex breaks a store. Changes the entire component palette.
Black or white text on the brand colour — computed, or fixed?	Accessibility floor. If computed, the design must look deliberate with either.
One typeface for all storefronts, or does the tenant get a choice?	Strong recommendation: one. A choice multiplies load cost and design risk for negligible differentiation.
Categories on the storefront — ship or defer?	Categories exist in the database but appear nowhere in the storefront. Adding navigation changes the catalogue page's whole structure.
Search on the catalogue?	Arguably unnecessary at thirty products; necessary the moment a client uploads three hundred.
Image gallery on the product page?	The data supports multiple images today. Fashion and beauty tenants will need it.
Does the storefront need a dark mode?	The library is installed. My instinct is no — it doubles the brand-colour problem for little gain on a shopping surface.
Is "store not found" neutral or a platform surface?	Small page, small free marketing opportunity.

Current state map

WHAT EXISTS AS OF TODAY

So you know what you're redesigning versus designing from nothing. Everything below is functional — the checkout has been tested end to end against the real payment gateway — but the visual layer is deliberately minimal, waiting on this brief.

SURFACE	ROUTE	STATE
Marketing home	/	PLACEHOLDER
Pricing	/pricing	NOT BUILT
Sign in / Sign up	/sign-in · /sign-up	CLERK DEFAULT
Catalogue	/	MINIMAL GRID
Product detail	/products/[slug]	ONE IMAGE
Cart	/cart	BUILT
Checkout	/checkout	BUILT
Payment overlay	—	PAYSTACK
Order confirmation	/order/[reference]	BUILT, 3 STATES
Store not found	/tenant-not-found	BARE
Empty-store state	—	ONE LINE OF TEXT
Missing-image placeholder	—	NOT BUILT
Stock-shortfall message	—	NOT BUILT

Two seeded demo stores exist and can be browsed end to end, including a real test payment. If it helps to see the current state in a browser rather than in screenshots, that's a five-minute setup and I'd recommend it before you start — particularly the checkout-to-confirmation stretch, which is hard to judge statically.

Written against the working build. Routes, field names and states above are taken from the code as it stands, not from a plan — where something is marked as not built, it genuinely doesn't exist yet, and where something is marked built, you can go and look at it.